המילון של גל

כל המושגים החדשים בעברית פשוטה

שיעורי גל: Git וכלי JavaScript • AI מודרני • Node.js • TypeScript ו־npm

המטרה: לא לשנן אנגלית. להבין מה כל מילה אומרת, מתי משתמשים בה, והאם זו פקודה קבועה או שם שהמתכנת המציא.

חשוב: המסמך מסביר רק מושגים שהופיעו בקוד, ב־README או בתמלולים של השיעורים החדשים הזמינים עד 2.9.2026. הוא אינו מוסיף חומר מתקדם שלא הופיע.

איך לקרוא כל דוגמת קוד

סוג	איך מזהים
מילה קבועה	חלק מהשפה או מהכלי. כותבים בדיוק, למשל function, type, npm install.
שם שהמתכנת בוחר	אפשר לבחור שם אחר, למשל customerName, Employee או compile.
ערך שהמתכנת בוחר	המידע עצמו, למשל 20, true או "Maya".
סימן קבוע	סימן בעל משמעות בשפה, למשל :, ?, |, &, => או {}.

מפת הלימוד

הסדר המומלץ למתחיל:

1. קודם להבין TypeScript וטיפוסים בסיסיים.
2. אחר כך להבין את מבנה הפרויקט: src, dist, package.json ו־tsconfig.json.
3. לאחר מכן ללמוד את פקודות npm ו־tsc.
4. רק אז לעבור לטיפוסים המתקדמים: union, type alias, optional ו־intersection.
5. בנפרד: Git, כלי AI ו־JavaScript מודרני מהשיעור הקודם.

חלק א׳ — יסודות TypeScript

אלה המושגים החשובים ביותר מהשיעור. כדאי להבין אותם לפני כל השאר.

1. TypeScript — טייפסקריפט

בפשטות: JavaScript בתוספת מערכת שבודקת אילו סוגי ערכים מותר להכניס לקוד.

מתי משתמשים: כשאנחנו רוצים לגלות טעויות עוד בזמן הכתיבה, לפני שהתוכנית רצה.

מה קבוע ומה בוחרים: TypeScript הוא שם השפה. קובץ המקור מסתיים ב־.ts.

```
const price: number = 19.9;
```

2. Type / Typing — טיפוס / הגדרת טיפוסים

בפשטות: תיאור של סוג המידע: טקסט, מספר, אמת/שקר או מבנה מורכב.

מתי משתמשים: כשמגדירים משתנה, פרמטר, תוצאת פונקציה או אובייקט.

מה קבוע ומה בוחרים: שם הטיפוס המובנה קבוע; שם המשתנה נבחר.

```
const customerName: string = "Maya";
```

3. Type safety — בטיחות טיפוסים

בפשטות: TypeScript מונעת מאיתנו להשתמש בערך מהסוג הלא נכון.

מתי משתמשים: כדי לגלות למשל שנשלח טקסט לפונקציה שמחכה למספר.

מה קבוע ומה בוחרים: זה מושג, לא פקודה שכותבים בקוד.

```
calcSubtotal("20", 3); // TypeScript error
```

4. Static check — בדיקה לפני הריצה

בפשטות: העורך או הקומפיילר בודקים את הקוד בלי להפעיל אותו.

מתי משתמשים: בזמן כתיבת TypeScript ובזמן הפעלת tsc.

מה קבוע ומה בוחרים: זה מושג. הודעת השגיאה מגיעה מ־TypeScript.

שימו לב: גל הדגיש: זו אינה בדיקת Unit Test ואינה בדיקת Runtime.

5. Runtime — זמן ריצה

בפשטות: הרגע שבו JavaScript באמת פועלת בדפדפן או ב־Node.

מתי משתמשים: כשבודקים מה קורה עם מידע אמיתי שהגיע ממשתמש או משרת.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

שימו לב: בדיקת TypeScript לא מחליפה Validation בזמן ריצה. גל אמר ששני הדברים אינם באים אחד על חשבון השני.

6. string — מחרוזת / טקסט

בפשטות: טיפוס של מילים ותווים.

מתי משתמשים: לשמות, אימייל, מזהה טקסטואלי או משפט קבלה.

מה קבוע ומה בוחרים: string קבוע; שם המשתנה והטקסט נבחרים.

```
const productName: string = "Notebook";
```

7. number — מספר

בפשטות: טיפוס אחד לכל המספרים ב־TypeScript: שלמים ועשרוניים.

מתי משתמשים: למחיר, כמות, גיל, אחוז ומזהה מספרי.

מה קבוע ומה בוחרים: number קבוע; שם המשתנה והמספר נבחרים.

```
const unitPrice: number = 19.9;
```

8. boolean — אמת או שקר

בפשטות: טיפוס שמכיל רק true או false.

מתי משתמשים: לשאלות כן/לא, למשל האם הלקוח חבר או האם המוצר שולם.

מה קבוע ומה בוחרים: boolean, true ו־false קבועים; שם המשתנה נבחר.

```
const isMember: boolean = true;
```

9. null — אין ערך בכוונה

בפשטות: ערך שמסמן שבכוונה אין כרגע מידע.

מתי משתמשים: כאשר רוצים לייצג חוסר ערך בצורה מפורשת.

מה קבוע ומה בוחרים: null היא מילה קבועה.

```
let selectedUser: User | null = null;
```

10. undefined — עדיין לא הוגדר

בפשטות: ערך שמופיע כאשר אין ערך או ששדה לא קיים.

מתי משתמשים: למשל כאשר קוראים שדה אופציונלי שלא הוכנס.

מה קבוע ומה בוחרים: undefined היא מילה קבועה.

```
employee.phone // may be undefined
```

11. any — כל סוג

בפשטות: מכבה כמעט את בדיקת הטיפוסים עבור אותו ערך.

מתי משתמשים: רק במצב חריג וזמני כאשר עדיין לא יודעים את הטיפוס.

מה קבוע ומה בוחרים: any קבוע.

```
let data: any;
```

שימו לב: גל אמר שבדרך כלל לא משתמשים ב־any ושזו דרך לא טובה, כי היא מבטלת את היתרון של TypeScript.

12. Type annotation — כתיבת הטיפוס ליד השם

בפשטות: הנקודתיים והטיפוס שאחריהם אומרים ל־TypeScript איזה ערך מותר.

מתי משתמשים: כשגל מבקש טיפוס מפורש למשתנה או לפרמטר.

מה קבוע ומה בוחרים: : והטיפוס קבועים; variableName הוא שם נבחר.

```
const variableName: number = 10;
```

13. Type inference — הסקת טיפוס אוטומטית

בפשטות: TypeScript מבינה את הטיפוס לפי הערך גם אם לא כתבנו אותו.

מתי משתמשים: כשערך ההתחלה ברור, למשל const price = 20.

מה קבוע ומה בוחרים: זה מושג. שם המשתנה והערך נבחרים.

```
const price = 20; // inferred as number
```

חלק ב׳ — טיפוסים בפונקציות ובאובייקטים

כאן מתחילים לראות יותר סימנים. בכל דוגמה חשוב לזהות מה קבוע ומה גל או המתכנת בחרו.

14. Parameter type — טיפוס של פרמטר

בפשטות: קובע איזה סוג ערך מותר להכניס לפונקציה.

מתי משתמשים: בכל פרמטר של פונקציית TypeScript בתרגיל.

מה קבוע ומה בוחרים: שם הפרמטר נבחר; : number קבוע לפי הסוג הדרוש.

```
function isAdult(age: number): boolean {
  return age >= 18;
}
```

15. Return type — טיפוס הערך שחוזר

בפשטות: הטיפוס שכתוב אחרי סוגרי הפרמטרים וקובע מה הפונקציה חייבת להחזיר.

מתי משתמשים: כאשר מוסיפים טיפוסים לפונקציה.

מה קבוע ומה בוחרים: הנקודתיים והטיפוס קבועים; גוף הפונקציה נכתב לפי המשימה.

```
function calcSubtotal(price: number, quantity: number): number {
  return price * quantity;
}
```

16. void — לא מחזירה ערך

בפשטות: טיפוס תוצאה לפונקציה שמבצעת פעולה אבל אינה מחזירה מידע בעזרת return.

מתי משתמשים: למשל פונקציה שמדפיסה או מעדכנת את הדף.

מה קבוע ומה בוחרים: void קבוע; שם הפונקציה נבחר.

```
function init(): void {
  console.log("start");
}
```

17. Object type — טיפוס של אובייקט

בפשטות: תבנית שמפרטת אילו שדות צריכים להיות באובייקט ומה הטיפוס של כל שדה.

מתי משתמשים: כאשר רוצים לוודא שלכל עובד יש name, id ו־email.

מה קבוע ומה בוחרים: שמות השדות והטיפוס המותאם נבחרים; התחביר { field: type } קבוע.

```
type Employee = {
  name: string;
  id: number;
  email: string;
};
```

18. Nested object — אובייקט בתוך אובייקט

בפשטות: שדה אחד מכיל אובייקט נוסף עם שדות משלו.

מתי משתמשים: למשל כתובת בתוך חברה או עובד.

מה קבוע ומה בוחרים: שמות address, city וכדומה נבחרים; מבנה הסוגריים קבוע.

```
address: { street: string; city: string };
```

19. Array<T> — מערך של טיפוס מסוים

בפשטות: אומר שכל האיברים במערך צריכים להתאים לטיפוס שבין < >.

מתי משתמשים: למשל מערך עובדים או מערך מזהים.

מה קבוע ומה בוחרים: Array והסוגריים הזוויתיים קבועים; Employee הוא טיפוס שהמתכנת הגדיר.

```
const employees: Array<Employee> = [];
```

20. Generic angle brackets < > — סוגריים זוויתיים של טיפוס

בפשטות: ב־Array<Employee> הם אומרים מהו טיפוס האיברים שבתוך המערך.

מתי משתמשים: כאשר משתמשים בטיפוס כללי כמו Array או ReturnType.

מה קבוע ומה בוחרים: הסימנים < > קבועים; הטיפוס שבתוכם משתנה לפי הצורך.

```
Array<number | string>
```

21. Optional property — שדה לא חובה ?

בפשטות: סימן שאלה אחרי שם שדה אומר שמותר לאובייקט להגיע גם בלי השדה הזה.

מתי משתמשים: כאשר למשל לא לכל עובד יש phone או address.

מה קבוע ומה בוחרים: ? קבוע; phone הוא שם שדה נבחר.

```
phone?: string;
```

22. Union — או |

בפשטות: הערך יכול להתאים לטיפוס אחד או לטיפוס אחר.

מתי משתמשים: כאשר מזהה יכול להיות number או string, או כאשר status מוגבל לכמה אפשרויות.

מה קבוע ומה בוחרים: הסימן | קבוע; הטיפוסים והאפשרויות נבחרים.

```
function addId(id: number | string): void {}
```

23. Literal union — רשימת ערכים מותרים

בפשטות: לא רק סוג טקסט, אלא טקסטים מסוימים בלבד.

מתי משתמשים: כאשר status חייב להיות אחת מכמה אפשרויות קבועות.

מה קבוע ומה בוחרים: | והמירכאות קבועים; המילים active וכו׳ נבחרות לפי הדרישה.

```
status: 'active' | 'inactive' | 'closed';
```

24. Intersection — וגם &

בפשטות: הערך חייב לכלול את הדרישות של שני טיפוסים יחד.

מתי משתמשים: כאשר משתמש צריך להיות גם User וגם Employee.

מה קבוע ומה בוחרים: & קבוע; שמות הטיפוסים נבחרים.

```
type UserType = User & Employee;
```

25. Type alias — שם לטיפוס מותאם

בפשטות: נותנים שם לתיאור של מבנה או שילוב טיפוסים כדי להשתמש בו שוב.

מתי משתמשים: כאשר מבנה חוזר או מורכב, למשל Employee או Company.

מה קבוע ומה בוחרים: type קבוע; Employee ושמות השדות נבחרים.

```
type Employee = { name: string; id: number };
```

שימו לב: בתרגילי Primitive Types גל כתב במפורש לא להוסיף עדיין type alias או interface. הם הופיעו בהדגמה מאוחרת יותר, לא בשלב הראשון של התרגיל.

26. Custom type — טיפוס שהמתכנת יצר

בפשטות: טיפוס שאינו string/number/boolean אלא תיאור שמתאים לבעיה שלנו.

מתי משתמשים: למשל טיפוס Employee לפרטי עובד.

מה קבוע ומה בוחרים: התחביר קבוע; שם הטיפוס וכל השדות נבחרים.

```
type Employee = { name: string; email: string };
```

27. typeof in a type — יצירת טיפוס מערך קיים

בפשטות: TypeScript מסתכלת על המבנה של ערך קיים ובונה ממנו טיפוס.

מתי משתמשים: כאשר כבר יש אובייקט, למשל JSON של מדינה, ורוצים טיפוס באותו מבנה.

מה קבוע ומה בוחרים: typeof קבוע; country הוא שם משתנה נבחר.

```
type Country = typeof country;
```

28. ReturnType — טיפוס התוצאה של פונקציה

בפשטות: כלי מובנה שמחלץ את הטיפוס שפונקציה מחזירה.

מתי משתמשים: כאשר רוצים להשתמש בטיפוס התוצאה בלי לכתוב אותו שוב ידנית.

מה קבוע ומה בוחרים: ReturnType ו־typeof קבועים; שם הפונקציה נבחר.

```
type CountResult = ReturnType<typeof getEmployeeCount>;
```

29. Function signature — חתימת פונקציה

בפשטות: מתארת אילו פרמטרים פונקציה מקבלת ומה היא מחזירה, בלי לכתוב את הפעולה עצמה.

מתי משתמשים: כאשר שדה בתוך אובייקט צריך להכיל פונקציה.

מה קבוע ומה בוחרים: הסוגריים, => והטיפוס קבועים; שם השדה נבחר.

```
getEmployeeCount: () => number;
```

30. Type assertion / as — אמירה ל־TypeScript: תתייחס לזה כטיפוס

בפשטות: אנחנו מבטיחים ל־TypeScript שהערך מתאים לטיפוס, גם אם היא לא הצליחה להוכיח זאת.

מתי משתמשים: רק כאשר אנחנו באמת יודעים את מבנה הערך והיא אינה יודעת.

מה קבוע ומה בוחרים: as קבוע; Company הוא טיפוס נבחר.

```
return {} as Company;
```

שימו לב: as אינו יוצר את השדות, אינו בודק מידע בזמן ריצה ועלול להסתיר טעות. גל אמר במפורש שהאובייקט הריק בדוגמה אינו דרך נכונה; הוא השתמש בו רק כדי להדגים casting.

31. Casting — המרת/כפיית התייחסות לטיפוס

בפשטות: השם המקצועי לפעולה שגל הדגים עם as.

מתי משתמשים: כאשר צריך לומר למערכת הטיפוסים איך להתייחס לערך.

מה קבוע ומה בוחרים: זה מושג; בדוגמה התחביר הוא as Company.

32. interface — חוזה למבנה נתונים

בפשטות: דרך נוספת ב־TypeScript לתאר אילו שדות יש באובייקט.

מתי משתמשים: בהמשך, כאשר המרצה ילמד ממשקי טיפוסים.

מה קבוע ומה בוחרים: interface קבוע; שם הממשק והשדות נבחרים.

שימו לב: המושג הוזכר, אבל גל ביקש לא להשתמש בו בתרגילי ה־Primitive Types הראשונים.

33. enum — קבוצת אפשרויות בעלות שמות

בפשטות: אפשרות TypeScript להגדיר רשימת ערכים קשורה.

מתי משתמשים: רק אם המרצה ילמד וידרוש זאת בהמשך.

מה קבוע ומה בוחרים: enum קבוע; שם הרשימה והערכים נבחרים.

שימו לב: enum רק הוזכר כהבדל בין JavaScript ל־TypeScript; הוא לא נדרש בתרגילים שנבדקו.

חלק ג׳ — Node.js, npm וחבילות

המילים האלה אינן פקודות JavaScript רגילות. הן שייכות לסביבת העבודה ולניהול הפרויקט.

34. Compiler / tsc — הקומפיילר של TypeScript

בפשטות: תוכנית שקוראת קובצי .ts ובונה מהם קובצי .js.

מתי משתמשים: אחרי כתיבת TypeScript ולפני שהדפדפן מריץ את הקוד.

מה קבוע ומה בוחרים: tsc היא פקודה קבועה.

```
tsc
```

35. Compile / Transpile — תרגום הקוד

בפשטות: העברת הקוד מ־TypeScript ל־JavaScript. גל אמר שהמונח המדויק יותר הוא transpile, אבל בשיעור הוא משתמש גם במילה compile.

מתי משתמשים: כאשר רוצים ליצור JavaScript שאפשר להריץ.

מה קבוע ומה בוחרים: אלה מושגים; הפקודה בפועל היא tsc.

```
src/index.ts  ->  tsc  ->  dist/index.js
```

36. Node.js — סביבת הרצת JavaScript מחוץ לדפדפן

בפשטות: תוכנה שמאפשרת להריץ JavaScript במחשב ומגיעה עם npm.

מתי משתמשים: להפעלת כלי פיתוח כמו TypeScript ולפעמים להרצת JS בטרמינל.

מה קבוע ומה בוחרים: node הוא שם הכלי והפקודה.

```
node -v
```

37. npm — מנהל החבילות של Node

בפשטות: כלי שמתקין ספריות ומריץ פקודות שמוגדרות בפרויקט.

מתי משתמשים: כאשר מתחילים פרויקט, מתקינים חבילה או מריצים script.

מה קבוע ומה בוחרים: npm הוא שם קבוע. שמות החבילות וה־script משתנים.

```
npm install
```

38. Package / Library — חבילה / ספריית קוד

בפשטות: קוד שמישהו אחר כתב ואפשר להתקין ולהשתמש בו בפרויקט.

מתי משתמשים: כאשר לא רוצים לבנות כל יכולת מאפס.

מה קבוע ומה בוחרים: זה מושג; שם החבילה נבחר, למשל axios.

39. Dependency — תלות של הפרויקט

בפשטות: חבילה שהפרויקט צריך כדי לעבוד.

מתי משתמשים: כאשר מתקינים ספרייה שהקוד משתמש בה בזמן ריצה.

מה קבוע ומה בוחרים: dependencies הוא שם שדה קבוע ב־package.json; שמות החבילות נבחרים.

```
npm i axios
```

40. devDependency — תלות לפיתוח בלבד

בפשטות: חבילה שנחוצה בזמן הכתיבה או הבנייה, אבל לא להפעלת המוצר הסופי.

מתי משתמשים: למשל TypeScript שמתרגמת את הקוד לפני העלאה.

מה קבוע ומה בוחרים: devDependencies קבוע ב־package.json; שם החבילה נבחר.

```
npm i typescript --save-dev
```

41. Production — סביבת המוצר האמיתי

בפשטות: הגרסה שהמשתמשים מפעילים, בניגוד לסביבת הפיתוח.

מתי משתמשים: כאשר מסבירים אילו חבילות צריכות לעלות עם המוצר.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

42. Development — סביבת הפיתוח

בפשטות: השלב שבו אנחנו כותבים, בודקים ומקמפלים את הקוד.

מתי משתמשים: בזמן העבודה שלנו כמפתחים.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

43. Global install -g — התקנה לכל המחשב

בפשטות: הפקודה זמינה בכל פרויקט במחשב.

מתי משתמשים: כאשר גל רוצה שהפקודה tsc תעבוד מכל תיקייה.

מה קבוע ומה בוחרים: -g הוא flag קבוע; שם החבילה נבחר.

```
npm i typescript -g
```

44. Local install — התקנה בתוך פרויקט אחד

בפשטות: החבילה נשמרת עבור התיקייה הנוכחית בלבד.

מתי משתמשים: כאשר רוצים שלכל פרויקט תהיה גרסה משלו.

מה קבוע ומה בוחרים: זה מושג; npm i typescript --save-dev מבצע התקנה מקומית.

45. Flag — אפשרות נוספת לפקודה

בפשטות: חלק שמתחיל בדרך כלל במקף ומשנה את אופן הפעולה של הפקודה.

מתי משתמשים: למשל כדי להתקין גלובלית או כחבילת פיתוח.

מה קבוע ומה בוחרים: ה־flag קבוע; למשל -g או --save-dev.

```
npm i typescript --save-dev
```

46. Axios — ספרייה לבקשות HTTP

בפשטות: ספרייה שעוזרת לשלוח בקשות לשרת, בדומה ל־Ajax GET שלמדתם.

מתי משתמשים: בהמשך, כאשר צריך להביא מידע מ־API.

מה קבוע ומה בוחרים: axios הוא שם חבילה קבוע.

```
npm i axios
```

שימו לב: בשיעור היא הותקנה והוסברה, אך בקוד שנבדק עדיין לא נעשה בה שימוש ממשי.

47. HTTP request — בקשה לשרת

בפשטות: הודעה שהתוכנית שולחת לשרת כדי לקבל או לשלוח מידע.

מתי משתמשים: למשל קבלת רשימת מוצרים מ־API.

מה קבוע ומה בוחרים: זה מושג. GET הוא אחד מסוגי הבקשות.

48. Open source — קוד פתוח

בפשטות: קוד שאנשים מפרסמים כדי שאחרים יוכלו להשתמש בו ולתרום לו.

מתי משתמשים: כאשר מתקינים חבילות ציבוריות דרך npm.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

חלק ד׳ — הקבצים והתיקיות בפרויקט

אם מבינים מי נכתב ביד ומי נוצר אוטומטית, מבנה הפרויקט נעשה הרבה פחות מפחיד.

49. package.json — תעודת הזהות של הפרויקט

בפשטות: קובץ שמכיל שם, גרסה, scripts ורשימת חבילות.

מתי משתמשים: אחרי npm init ובכל פרויקט Node/npm.

מה קבוע ומה בוחרים: שם הקובץ והשדות המרכזיים קבועים; שם הפרויקט וה־scripts נבחרים.

```
"scripts": { "compile": "tsc --watch" }
```

50. package-lock.json — נעילת גרסאות החבילות

בפשטות: קובץ שנוצר אוטומטית ושומר אילו גרסאות מדויקות הותקנו.

מתי משתמשים: npm יוצרת ומעדכנת אותו בזמן התקנה.

מה קבוע ומה בוחרים: שם הקובץ קבוע. בדרך כלל לא עורכים אותו ביד.

51. node_modules — תיקיית החבילות המותקנות

בפשטות: תיקייה גדולה שמכילה את הקוד של כל החבילות והתלויות שלהן.

מתי משתמשים: נוצרת אחרי npm install.

מה קבוע ומה בוחרים: שם התיקייה קבוע. לא עורכים את הקבצים בתוכה.

שימו לב: אם התיקייה חסרה, מריצים npm install. בדרך כלל לא מעלים node_modules ל־Git.

52. tsconfig.json — הגדרות TypeScript

בפשטות: קובץ שאומר לקומפיילר מאיפה לקרוא, לאן לכתוב ואילו בדיקות להפעיל.

מתי משתמשים: בפרויקט TypeScript, אחרי tsc --init.

מה קבוע ומה בוחרים: שם הקובץ ושמות האפשרויות קבועים; הערכים מותאמים לפרויקט.

53. src — Source — קוד המקור

בפשטות: התיקייה שבה אנחנו כותבים את קובצי TypeScript.

מתי משתמשים: בזמן פיתוח.

מה קבוע ומה בוחרים: src הוא שם מקובל שהמרצה בחר במבנה הפרויקט; הוא אינו פקודת שפה.

54. dist — Distribution — הקוד שנוצר

בפשטות: התיקייה שאליה tsc כותבת JavaScript מוכן להרצה.

מתי משתמשים: אחרי קומפילציה.

מה קבוע ומה בוחרים: dist הוא שם מקובל שהוגדר ב־outDir; לא עורכים את ה־JS שנוצר ביד.

55. rootDir — תיקיית המקור לקומפילציה

בפשטות: אומרת ל־TypeScript איפה נמצאים קובצי .ts.

מתי משתמשים: בתוך compilerOptions ב־tsconfig.json.

מה קבוע ומה בוחרים: rootDir קבוע; src/. הוא נתיב שהמתכנת בוחר.

```
"rootDir": "./src"
```

56. outDir — תיקיית הפלט

בפשטות: אומרת איפה לשמור את קובצי .js שנוצרו.

מתי משתמשים: בתוך tsconfig.json.

מה קבוע ומה בוחרים: outDir קבוע; dist/. הוא נתיב נבחר.

```
"outDir": "./dist"
```

57. strict — בדיקות טיפוסים מחמירות

בפשטות: מפעילה קבוצה של בדיקות שעוזרות למצוא יותר טעויות.

מתי משתמשים: בפרויקט TypeScript שבו רוצים בדיקה חזקה.

מה קבוע ומה בוחרים: strict קבוע; true מפעיל.

```
"strict": true
```

58. noEmitOnError — לא ליצור JS כשיש שגיאה

בפשטות: אם TypeScript מוצאת שגיאה, היא לא מוציאה קובץ JavaScript חדש.

מתי משתמשים: כדי לא להריץ בטעות פלט שנבנה מקוד שגוי.

מה קבוע ומה בוחרים: noEmitOnError קבוע; true מפעיל.

```
"noEmitOnError": true
```

59. include — מה כן לכלול

בפשטות: רשימה של קבצים או תיקיות שהקומפיילר צריך לבדוק.

מתי משתמשים: ב־tsconfig.json כאשר רוצים להגביל את הקומפילציה.

מה קבוע ומה בוחרים: include קבוע; הנתיבים נבחרים.

```
"include": ["./**/*.ts"]
```

60. exclude — ממה להתעלם

בפשטות: רשימת תיקיות שלא צריך לקמפל.

מתי משתמשים: כדי לדלג על node_modules ועל קוד פלט.

מה קבוע ומה בוחרים: exclude קבוע; שמות התיקיות נבחרים.

```
"exclude": ["node_modules", "dist"]
```

61. target — גרסת JavaScript לפלט

בפשטות: קובעת לאיזו רמת JavaScript TypeScript תתרגם.

מתי משתמשים: כאשר צריך להתאים לסביבה שתפעיל את הקוד.

מה קבוע ומה בוחרים: target קבוע; ES2020 או esnext הם ערכים נבחרים.

```
"target": "ES2020"
```

62. script in package.json — קיצור דרך לפקודה

בפשטות: שם נוח שנותנים לפקודת טרמינל ארוכה.

מתי משתמשים: כדי להריץ npm run compile במקום לזכור את הפקודה המלאה.

מה קבוע ומה בוחרים: scripts ו־npm run קבועים; compile הוא שם שהמתכנת בחר.

```
"compile": "tsc --watch"
```

63. Source of truth — המקור הקובע

בפשטות: המקום שבו נמצאת הגרסה שעל פיה כולם עובדים.

מתי משתמשים: בשיעורי גל: ה־Repository וה־README הם המקור לחומר ולהוראות.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

חלק ה׳ — פקודות הטרמינל

את הפקודות האלה מקלידים בטרמינל, לא בתוך קובץ JavaScript או TypeScript.

64. node -v — בדיקת גרסת Node

בפשטות: מציגה אם Node מותקנת ואיזו גרסה קיימת.

מתי משתמשים: אחרי התקנה או כאשר פקודת Node אינה עובדת.

מה קבוע ומה בוחרים: הפקודה קבועה; לא מכניסים לתוכה שם משלנו.

```
node -v
```

65. npm -v — בדיקת גרסת npm

בפשטות: מציגה את גרסת npm.

מתי משתמשים: אחרי התקנת Node.

מה קבוע ומה בוחרים: הפקודה קבועה.

```
npm -v
```

66. npm init — התחלת פרויקט npm

בפשטות: שואלת שאלות ויוצרת package.json בתיקייה הנוכחית.

מתי משתמשים: פעם אחת בתחילת פרויקט חדש.

מה קבוע ומה בוחרים: npm init קבוע; התשובות לשאלות נבחרות.

```
npm init
```

67. npm install — התקנת כל החבילות

בפשטות: קוראת את package.json ומתקינה את התלויות לתוך node_modules.

מתי משתמשים: אחרי clone או כאשר node_modules חסרה.

מה קבוע ומה בוחרים: הפקודה קבועה.

```
npm install
```

68. npm install package — התקנת חבילה מסוימת

בפשטות: מתקינה חבילה ומוסיפה אותה ל־package.json.

מתי משתמשים: כאשר הפרויקט צריך ספרייה חדשה.

מה קבוע ומה בוחרים: npm install קבוע; package מוחלף בשם החבילה.

```
npm install axios
```

69. npm i — קיצור של npm install

בפשטות: אותה פעולה בכתיבה קצרה יותר.

מתי משתמשים: כאשר רוצים להתקין חבילה או את כל החבילות.

מה קבוע ומה בוחרים: npm i קבוע; שם החבילה נבחר.

```
npm i axios
```

70. tsc --init — יצירת tsconfig

בפשטות: יוצרת קובץ tsconfig.json התחלתי.

מתי משתמשים: פעם אחת בתחילת פרויקט TypeScript.

מה קבוע ומה בוחרים: הפקודה קבועה.

```
tsc --init
```

71. tsc — קומפילציה פעם אחת

בפשטות: קוראת את tsconfig ומתרגמת את קובצי TypeScript ל־JavaScript.

מתי משתמשים: כשרוצים לבנות את הקוד פעם אחת.

מה קבוע ומה בוחרים: הפקודה קבועה.

```
tsc
```

72. tsc --watch — קומפילציה רציפה

בפשטות: נשארת פתוחה ומקמפלת מחדש אחרי כל שמירה.

מתי משתמשים: בזמן תרגול ועבודה שוטפת.

מה קבוע ומה בוחרים: tsc ו־watch-- קבועים.

```
tsc --watch
```

73. npm run compile — הרצת קיצור בשם compile

בפשטות: npm מחפשת את compile בתוך scripts ב־package.json ומריצה את הפקודה ששמורה שם.

מתי משתמשים: כאשר הוגדר script בשם compile.

מה קבוע ומה בוחרים: npm run קבוע; compile הוא שם שהמתכנת בחר.

```
npm run compile
```

74. Current directory — התיקייה שבה הטרמינל עומד

בפשטות: כל פקודה פועלת ביחס לתיקייה הנוכחית.

מתי משתמשים: לפני npm init, npm install או tsc צריך לוודא שאנחנו בתיקיית הפרויקט הנכונה.

מה קבוע ומה בוחרים: זה מושג; הנתיב משתנה לפי המחשב.

75. cd — מעבר תיקייה בטרמינל

בפשטות: משנה את התיקייה הנוכחית.

מתי משתמשים: כאשר צריך להיכנס לפרויקט או ל־src.

מה קבוע ומה בוחרים: cd קבוע; הנתיב אחריו נבחר.

```
cd src
```

חלק ו׳ — Git וכלי AI

המילים האלה מתארות את דרך העבודה החדשה של גל. הן אינן כולן פקודות JavaScript.

76. Git — מערכת ניהול גרסאות

בפשטות: שומרת היסטוריה של שינויים בקוד ומאפשרת לעבוד יחד.

מתי משתמשים: בכל פרויקט כדי לתעד ולסנכרן שינויים.

מה קבוע ומה בוחרים: Git הוא שם הכלי; שמות commits ו־branches נבחרים.

77. GitHub — אתר לאחסון מאגרי Git

בפשטות: מקום מרוחק שבו נשמר ה־Repository ואפשר לשתף אותו.

מתי משתמשים: כדי להעלות, להוריד ולשתף קוד.

מה קבוע ומה בוחרים: זה שם השירות, לא פקודת קוד.

78. Repository / Repo — מאגר

בפשטות: תיקיית פרויקט יחד עם היסטוריית Git שלה.

מתי משתמשים: כאשר עובדים על קוד שמנוהל ב־Git.

מה קבוע ומה בוחרים: זה מושג; שם ה־Repo נבחר.

79. Clone — הורדת עותק ראשון

בפשטות: מעתיק Repository מרוחק למחשב ויוצר חיבור אליו.

מתי משתמשים: פעם ראשונה שמקבלים פרויקט קיים.

מה קבוע ומה בוחרים: git clone קבוע; הכתובת נבחרת.

```
git clone https://github.com/user/repo.git
```

80. Pull — משיכת עדכונים

בפשטות: מוריד ומחבר למחשב את השינויים החדשים מה־Repository המרוחק.

מתי משתמשים: לפני עבודה או כאשר המרצה העלה חומר חדש.

מה קבוע ומה בוחרים: git pull קבוע.

```
git pull
```

81. Commit — נקודת שמירה מתועדת

בפשטות: אוסף שינויים ונותן להם הודעה המתארת מה נעשה.

מתי משתמשים: אחרי שסיימנו שינוי ברור ועובד.

מה קבוע ומה בוחרים: git commit -m קבוע; ההודעה נבחרת.

```
git commit -m "add TypeScript exercise"
```

82. Push — העלאת commits

בפשטות: מעלה את נקודות השמירה מהמחשב ל־GitHub.

מתי משתמשים: אחרי commit כשרוצים לשתף או לגבות.

מה קבוע ומה בוחרים: git push קבוע.

```
git push
```

83. Branch — ענף עבודה

בפשטות: קו עבודה נפרד שאפשר לשנות בו בלי לגעת מיד ב־main.

מתי משתמשים: כאשר מפתחים שינוי בנפרד.

מה קבוע ומה בוחרים: branch הוא מושג; שם הענף נבחר.

84. Merge — מיזוג

בפשטות: חיבור שינויים מענף אחד לענף אחר.

מתי משתמשים: אחרי שסיימו עבודה בענף ורוצים לצרף אותה.

מה קבוע ומה בוחרים: git merge קבוע; שם הענף נבחר.

85. **Diff** — הבדל בין גרסאות

בפשטות: מציג אילו שורות נוספו, נמחקו או השתנו.

מתי משתמשים: לפני commit או בזמן review.

מה קבוע ומה בוחרים: git diff קבוע.

```
git diff
```

86. **Conflict** — התנגשות

בפשטות: שני שינויים שונים נגעו באותו מקום ו־Git לא יודע לבד במה לבחור.

מתי משתמשים: בזמן merge או pull כאשר צריך להחליט ידנית.

מה קבוע ומה בוחרים: זה מושג, לא פקודה יחידה.

87. **Cursor** — עורך קוד עם כלי **AI**

בפשטות: סביבת עבודה דומה ל־VS Code שמאפשרת לשאול AI ולתת לו לבצע שינויים.

מתי משתמשים: כאשר לומדים, מתכננים או משנים פרויקט בעזרת AI.

מה קבוע ומה בוחרים: זה שם התוכנה.

88. **Prompt** — הבקשה ל־**AI**

בפשטות: הטקסט שאנחנו כותבים כדי להסביר מה אנחנו רוצים.

מתי משתמשים: בכל שיחה או משימה מול AI.

מה קבוע ומה בוחרים: זה מושג; תוכן הבקשה נבחר על ידינו.

89. **Context** — המידע שה־**AI** רואה

בפשטות: הקבצים, הקוד וההודעות שעליהם ה־AI מבסס את התשובה.

מתי משתמשים: כאשר רוצים תשובה שמתאימה באמת לפרויקט.

מה קבוע ומה בוחרים: זה מושג. אנחנו בוחרים אילו קבצים לצרף.

90. **Token** — יחידת טקסט של מודל **AI**

בפשטות: המודל קורא וכותב טקסט ביחידות קטנות; ככל שההקשר ארוך יותר משתמשים ביותר tokens.

מתי משתמשים: כדי להבין מגבלות אורך ועלות של מודל.

מה קבוע ומה בוחרים: זה מושג, לא פקודה.

91. **Model** — מודל ה־**AI**

בפשטות: המנוע שמייצר תשובות וקוד.

מתי משתמשים: כאשר בוחרים איזה AI לבצע משימה.

מה קבוע ומה בוחרים: זה מושג; שם המודל משתנה.

שימו לב: גל המליץ לא לקפוץ בין מודלים אקראיים באמצע פרויקט בלי סיבה.

92. **Ask mode** — מצב שאלות

בפשטות: ה־AI מסביר ומייעץ בלי לבצע שינוי גדול בקבצים.
מתי משתמשים: כשרוצים להבין קוד או לבדוק רעיון.
מה קבוע ומה בוחרים: Ask הוא שם מצב בכלי.

93. **Plan mode** — מצב תכנון

בפשטות: ה־AI בונה תוכנית עבודה לפני שמבצע.
מתי משתמשים: לפני משימה גדולה או שינוי בכמה קבצים.
מה קבוע ומה בוחרים: Plan הוא שם מצב.

שימו לב: גל הדגיש לתכנן לפני שנותנים ל־Agent לבצע שינוי גדול.

94. **Agent mode** — מצב ביצוע

בפשטות: ה־AI יכול לקרוא קבצים, לערוך ולהריץ פקודות.
מתי משתמשים: אחרי שהדרישה והתוכנית ברורות.
מה קבוע ומה בוחרים: Agent הוא שם מצב. עדיין חייבים לבדוק ולהבין את התוצאה.

95. **Rule** — כלל קבוע ל־**AI** בפרויקט

בפשטות: הוראה שנשמרת בפרויקט ומנחה את ה־AI בכל עבודה מתאימה.
מתי משתמשים: למשל כלל שמפריד HTML, CSS ו־JS.
מה קבוע ומה בוחרים: Rule הוא מושג; תוכן הכלל נכתב לפי הצורך.

96. **Skill** — תהליך עבודה חוזר ל־**AI**

בפשטות: הוראות מסודרות לביצוע משימה שחוזרת שוב ושוב.
מתי משתמשים: למשל סדר קבוע לבדיקות או להעלאה ל־Git.
מה קבוע ומה בוחרים: Skill הוא מושג; שם ותוכן ה־Skill נבחרים.

97. **Review** — בדיקת קוד

בפשטות: מבקשים לעבור על הקוד ולמצוא בעיות או אי־התאמות.
מתי משתמשים: אחרי כתיבה או המרה מ־JS ל־TS.
מה קבוע ומה בוחרים: זו פעולה/בקשה לכלי AI; הקבצים שנבדקים נבחרים.

חלק ז׳ — JavaScript מודרני, DOM ו־Debugging

חלק מהמושגים הופיעו בשיעור כלי ה־AI ובכללי Cursor של גל. הם מסבירים את סגנון הקוד החדש.

98. **ECMAScript** — התקן של **JavaScript**

בפשטות: השם הרשמי של המפרט שמגדיר איך JavaScript עובדת.
מתי משתמשים: כאשר מדברים על גרסאות וסגנון JavaScript מודרני.
מה קבוע ומה בוחרים: זה מושג, לא פקודה.

99. Arrow function => — פונקציית חץ

בפשטות: דרך קצרה לכתוב פונקציה.

מתי משתמשים: כאשר רוצים לשמור פונקציה במשתנה או להעביר אותה כערך.

מה קבוע ומה בוחרים: => קבוע; שם המשתנה והפרמטרים נבחרים.

```
const getUserName = (user: UserSystem) => user.name;
```

100. Template literal — מחרוזת עם backticks

בפשטות: טקסט בין ` ` שבתוכו אפשר לשלב ערכים באמצעות ${}.

מתי משתמשים: כאשר בונים הודעה עם משתנים.

מה קבוע ומה בוחרים: ה־backticks ו־${} קבועים; הטקסט ושם המשתנה נבחרים.

```
return `sku_${id}`;
```

101. Destructuring — פירוק ערכים

בפשטות: לקיחת שדות מאובייקט או איברים ממערך לתוך משתנים נפרדים.

מתי משתמשים: כאשר צריך כמה שדות מאותו אובייקט.

מה קבוע ומה בוחרים: הסוגריים והמבנה קבועים; שמות השדות תלויים באובייקט.

```
const { name, email } = employee;
```

102. Optional chaining ?. — גישה בטוחה לשדה

בפשטות: אם החלק שלפני ?. חסר, מתקבל undefined במקום שגיאה.

מתי משתמשים: כאשר אובייקט או שדה עלולים לא להיות קיימים.

מה קבוע ומה בוחרים: ?. קבוע; שמות האובייקט והשדה נבחרים.

```
employee.address?.city
```

103. Nullish coalescing ?? — ערך חלופי כשאין ערך

בפשטות: משתמש בערך הימני רק אם השמאלי הוא null או undefined.

מתי משתמשים: כאשר רוצים ברירת מחדל בלי להחליף 0 או מחרוזת ריקה.

מה קבוע ומה בוחרים: ?? קבוע; שני הערכים נבחרים.

```
const phone = employee.phone ?? "No phone";
```

104. querySelector — חיפוש אלמנט לפי selector

בפשטות: מחפש את האלמנט הראשון שמתאים ל־CSS selector.

מתי משתמשים: כאשר רוצים למצוא אלמנט ב־DOM לפי id, class או tag.

מה קבוע ומה בוחרים: querySelector קבוע; ה־selector בתוך המירכאות נבחר.

```
document.querySelector('#title')
```

105. querySelectorAll — חיפוש כל האלמנטים

בפשטות: מחזיר אוסף של כל האלמנטים שמתאימים ל־selector.

מתי משתמשים: כאשר רוצים לעבור על כמה אלמנטים.

מה קבוע ומה בוחרים: querySelectorAll קבוע; ה־selector נבחר.

```
document.querySelectorAll('.card')
```

106. createElement — יצירת אלמנט HTML ב־JS

בפשטות: יוצר אלמנט חדש בזיכרון לפני שמכניסים אותו לדף.

מתי משתמשים: כאשר בונים תצוגה דינמית לפי שיטת גל.

מה קבוע ומה בוחרים: createElement קבוע; שם התג נבחר.

```
const row = document.createElement('tr');
```

107. textContent — טקסט בתוך אלמנט

בפשטות: מכניס או קורא טקסט רגיל בלי לפרש אותו כ־HTML.

מתי משתמשים: כאשר רוצים להציג טקסט בבטחה.

מה קבוע ומה בוחרים: textContent קבוע; הערך נבחר.

```
cell.textContent = product.name;
```

108. append / prepend — הוספה לסוף / להתחלה

בפשטות: מכניסים אלמנט ילד לתוך אלמנט אחר.

מתי משתמשים: אחרי createElement כדי להוסיף את האלמנט ל־DOM.

מה קבוע ומה בוחרים: append ו־prepend קבועים; האלמנטים נבחרים.

```
tableBody.append(row);
```

109. DocumentFragment — מיכל זמני לאלמנטים

בפשטות: אוספים כמה אלמנטים מחוץ לדף ואז מוסיפים את כולם יחד.

מתי משתמשים: כאשר בונים הרבה שורות ורוצים לעדכן את ה־DOM בפעם אחת.

מה קבוע ומה בוחרים: DocumentFragment הוא API מובנה.

```
const fragment = document.createDocumentFragment();
```

110. new Date() — יצירת תאריך ושעה

בפשטות: יוצר אובייקט Date של הזמן הנוכחי.

מתי משתמשים: כאשר שומרים או מציגים מתי פעולה קרתה.

מה קבוע ומה בוחרים: new ו־Date קבועים.

```
const now = new Date();
```

111. toLocaleString() — תאריך בפורמט מקומי

בפשטות: מציג Date לפי השפה והאזור של הדפדפן.

מתי משתמשים: לתצוגה נוחה למשתמש.

מה קבוע ומה בוחרים: שם המתודה קבוע.

```
now.toLocaleString()
```

112. toISOString() — תאריך בפורמט ISO/UTC

בפשטות: מחזיר טקסט אחיד של זמן לפי UTC.

מתי משתמשים: לשמירה במסד נתונים או להעברת זמן בצורה אחידה.

מה קבוע ומה בוחרים: שם המתודה קבוע.

```
now.toISOString()
```

113. Debugging — איתור תקלה

בפשטות: עצירה, בדיקה והבנה של מה הקוד עושה בפועל.

מתי משתמשים: כאשר התוצאה שגויה או הקוד נעצר.

מה קבוע ומה בוחרים: זה תהליך, לא פקודה אחת.

114. Breakpoint — נקודת עצירה

בפשטות: סימון שורה שבה הדפדפן עוצר את ריצת הקוד.

מתי משתמשים: כדי לבדוק ערכי משתנים צעד אחר צעד.

מה קבוע ומה בוחרים: זה כלי Debugger, לא קוד JavaScript.

115. Scope — תחום גישה

בפשטות: קובע מאיפה בקוד אפשר לראות משתנה.

מתי משתמשים: כאשר משתנה זמין בתוך פונקציה או בלוק אבל לא מחוץ להם.

מה קבוע ומה בוחרים: זה מושג. const ו־let הם בעלי block scope.

116. Inline code — קוד בתוך HTML

בפשטות: JavaScript או CSS שנכתבים ישירות בתוך תגית HTML.

מתי משתמשים: רק אם הדרישה מתירה; בשיטת גל לפרויקט הזה מפרידים קבצים.

מה קבוע ומה בוחרים: זה מושג.

שימו לב: ב־Rule של גל כתוב להפריד HTML, CSS ו־JS ולא לכתוב קוד inline.

117. render / init — שמות נפוצים שהמתכנת בוחר

בפשטות: render בדרך כלל מציגה מידע; init בדרך כלל מתחילה את התוכנית. אלה אינם פקודות JavaScript.

מתי משתמשים: כאשר המתכנת בוחר שמות ברורים לפונקציות.

מה קבוע ומה בוחרים: השמות נבחרים ואפשר להחליפם, כל עוד מעדכנים את הקריאות.

```
function init() { render(); }
```

118. @deprecated — סימון שפונקציה ישנה

בפשטות: הערת תיעוד שאומרת שלא מומלץ להשתמש בפונקציה ויש חלופה חדשה.

מתי משתמשים: כאשר מתחזקים API או קוד ישן.

מה קבוע ומה בוחרים: @deprecated הוא תג תיעוד קבוע; ההסבר נבחר.

```
/** @deprecated Use newFunction() instead. */
```

שימו לב: המושג הופיע ב־dummy.ts כהדגמה. הוא אינו חלק מדרישת תרגילי ה־Primitive Types.

חלק ח׳ — מה גל ביקש לעשות ומה לא לעשות

רמת הכלל	ההוראה בפשטות
חובה	בתרגיל JS ל־TypeScript: להשאיר את אותה לוגיקה ורק להוסיף טיפוסים.
חובה	לתת טיפוס לפרמטרים, לערכי החזרה ולמשתנים בעלי שם, לפי דרישת התרגיל.
אסור בשלב הראשון	לא להוסיף interface, type alias או Object Types לתרגילי ה־Primitive Types.
חובה	לקמפל עם tsc עד שאין שגיאות, ואז לוודא שהפלט נשאר זהה ל־JavaScript המקורי.
מועדף	לכתוב TypeScript בתוך src ולהוציא JavaScript שנוצר אל dist.
אסור	לא לערוך ידנית את קובצי JavaScript שנוצרו ב־dist ולא את הקבצים בתוך node_modules.
לא מומלץ	לא להשתמש ב־any בדרך כלל, מפני שהוא מבטל את בדיקת הטיפוסים.
חשוב	בדיקת TypeScript היא לפני הריצה; היא אינה מחליפה Validation של מידע אמיתי בזמן ריצה.

רמת הכלל	ההוראה בפשטות
חשוב	אינו Validation as ואינו יוצר מידע חסר. הדוגמה return {} as Company הייתה הדגמה בלבד.
שיטת עבודה	ה־Repository וה־README הם מקור האמת לחומר ולהוראות.
שיטת עבודה	לתכנן לפני שמפעילים Agent על שינוי גדול, ולקרוא ולבדוק את הקוד שה־AI יצר.
שיטת קוד	להפריד HTML, CSS ו־JavaScript; לא להשתמש בקוד inline לפי ה־Rule של גל.
שינוי מהעבר	אצל גל מועדפים querySelector/createElement/addEventListener. זה שונה מסגנון מבחן שחר שבו הועדפו getElementById/innerHTML/onsubmit.

Workflow אחד שמחבר את הכול

שלב	מה עושים
1	נכנסים בטרמינל לתיקיית הפרויקט הנכונה.
2	אם זה פרויקט קיים: npm install. אם חדש: npm init.
3	כותבים TypeScript ב־src ומוסיפים טיפוסים לפי הדרישה.
4	מריצים tsc או npm run compile.
5	בודקת TypeScript שגיאות ויוצרת JavaScript ב־dist.
6	ה־HTML טוען את dist/index.js או dist/shop.js — לא את קובץ ה־ts.
7	מריצים ובודקים שההתנהגות והפלט נכונים.
8	בודקים git diff, מבצעים commit, ואז push כשמוכנים לשתף.

דוגמה שלמה קצרה מהשיעור

```
function calcSubtotal(unitPrice: number, quantity: number): number {
  return unitPrice * quantity;
}

const subtotal: number = calcSubtotal(19.9, 3);
```

פירוק:

function, return, הסוגריים והסוגריים המסולסלים — תחביר קבוע.
calcSubtotal, unitPrice, quantity — שמות שהמתכנת בחר ו־subtotal.
number : בדיקת TypeScript: הפרמטר או התוצאה חייבים להיות מספר — 19.9 ו־3 — ערכים שהמתכנת הכניס.
* — פעולת כפל של JavaScript.

אינדקס מהיר — איפה לחפש כל מילה

הקבוצה	חלק	מה מחפשים
שפה עם בדיקת טיפוסים	א׳	TypeScript
טיפוסים בסיסיים	א׳	string / number / boolean / any
טיפוסים בפונקציה	ב׳	parameter / return / void
טיפוסים מותאמים	ב׳	type alias / union / intersection / optional
כלי טיפוסים מתקדמים	ב׳	typeof / ReturnType / as
סביבת עבודה וחבילות	ג׳	Node / npm / dependency / Axios
מבנה הפרויקט	ד׳	package.json / tsconfig / src / dist
פקודות טרמינל	ה׳	npm install / tsc / npm run
ניהול גרסאות	ו׳	Git / clone / pull / commit / push
כלי AI	ו׳	Prompt / Context / Plan / Agent / Rule
JavaScript מודרני	ז׳	Arrow / Template literal / DOM
איתור תקלות	ז׳	Debugging / Breakpoint / Scope

מקורות שנבדקו

• מאגר גל FS_AI_45800_9, תיקיות AI-Tools-30-8 ו־Typescript-02-9, מעודכן עד commit 9697d41 מ־02.09.2026.
• README הראשי ו־Typescript/readme.md-02-9.
• קוד השיעור: dummy.ts, lab_2/src/type_alias.ts, lab_2/src/shop.ts, lab_1/src/index.ts וקובצי התרגילים.
• תמלול מלא של שיעור 02.09.2026: TYPESCRIPT-ESSENTIALS-2026-09-02-full-transcript.txt.
• מדריכי השיעור L-2026-08-30 ו־L-2026-09-02 במאגר הלימוד האישי.